

SPM PROJECT MODULE 1 REPORT

Project: Partition Mapping Kernel

Hamas Ali

Student ID: 726267

1. Mapping Function Design

In this project, we focus on the partition mapping step of a partitioned hash join, where each input key must be assigned to a partition in the range $[0, P)$. Since this operation is applied to all elements and lies on the critical path, its efficiency is essential for overall performance. The goal is to design a mapping function that is both computationally efficient and suitable for parallel execution.

The partition mapping is implemented using a **multiplicative hashing scheme with XOR-based bit folding**, designed to be efficient, deterministic, and suitable for SIMD optimization. For each 64-bit key k , the mapping first reduces the key to 32 bits by combining its lower and upper halves using a bitwise XOR operation, i.e., $x = l_o \oplus h_i$, where l_o and h_i denote the lower and upper 32 bits, respectively. This folding step ensures that information from the entire key contributes to the hash value while maintaining computational efficiency. The resulting value is then processed using multiplicative hashing, $h = x \times 0x9E3779B1$, where the constant is derived from Knuth's multiplicative method and helps achieve a uniform distribution of keys.

$$part_id = (h \gg (32 - \log_2(P))) \& (P - 1)$$

where P is the number of partitions. Choosing P as a power of two allows the modulo operation to be replaced by a simple bitmask, significantly reducing computational cost. This design uses only simple arithmetic and bitwise operations (XOR, multiplication, shift, and mask), avoids branching, and is therefore highly suitable for both compilers auto-vectorization and manual SIMD implementations. Additionally, the mapping is fully deterministic, ensuring reproducible results across different implementations.

2. Implementations

Two versions of the plain C++ implementation are generated from the same source file (src/main.cpp) using the provided Makefile.

2.1 Baseline (Scalar Version)

The baseline version is compiled using the flags:

-O3 -mavx2 -fno-tree-vectorize

In this version, -O3 enables compiler optimizations, while -fno-tree-vectorize disables automatic loop vectorization. This ensures that the mapping loop is executed in scalar form and can be used as a reference for performance comparison.

2.2 GCC Auto-Vectorized Version

The auto-vectorized version is compiled using:

-O3 -mavx2 -fopt-info-vec-all=reports/vectorization_report.txt

Here, the compiler is allowed to apply automatic vectorization. The additional flag generates a report that shows whether the loop was successfully vectorized.

2.3 Vectorization Evidence

Compiler flag used: **-O3 -mavx2 -fopt-info-vec-all=reports/vectorization_report.txt**.

Excerpt from GCC vectorization report (reports/vectorization_report.txt):

```
h.ali76@node09:~/Partition_hash_joined_Project/module_1$ grep -n "src/main.cpp:98" reports/vectorization_report.txt
98:src/main.cpp:98:22: optimized: loop vectorized using 32 byte vectors
99:src/main.cpp:98:22: optimized: loop vectorized using 16 byte vectors
701:src/main.cpp:98:22: optimized: loop vectorized using 32 byte vectors
702:src/main.cpp:98:22: optimized: loop vectorized using 16 byte vectors
```

The report confirms that the hot loop at line 98 of src/main.cpp. The map function in code was successfully vectorized by GCC using 32-byte (256-bit AVX2) vectors. The 16-byte variant is an additional fallback generated by the compiler. This confirms that the loop structure, which is free of data dependencies and annotated with #pragma GCC ivde, is suitable for auto-vectorization.

3. AVX2 Intrinsics Implementation

A manual SIMD implementation is developed using AVX2 intrinsics to further optimize the partition mapping kernel. In this version, multiple elements are processed in parallel using 256-bit vector registers. Specifically, each iteration handles 8 values at a time, since AVX2 can operate on eight 32-bit integers simultaneously.

The implementation follows the same logical steps as the scalar version to ensure correctness. First, each 64-bit key is reduced to a 32-bit value using the same XOR-based folding operation. This folding step is performed before loading the values into vector registers. The folded values are then processed using SIMD instructions: the multiplicative hash is computed using `_mm256_mullo_epi32`, followed by a right shift using `_mm256_srli_epi32`, and a masking operation using `_mm256_and_si256` to obtain the final partition identifiers in the range $[0, P)$. The computed results are stored back to memory using `_mm256_storeu_si256`.

To handle cases where the total number of elements is not a multiple of 8, a scalar tail loop is used to process the remaining elements. This ensures that the implementation works correctly for all input sizes. Importantly, the AVX2 version preserves the same mapping logic as the scalar implementation, guaranteeing identical output across all versions.

4. Correctness Verification

Correctness of the implementations is verified using two complementary methods:

4.1 Element-wise Comparison (Small Input)

A small input configuration is used to directly compare outputs element by element:

- $N = 16, P = 8, \text{seed} = 42$

All implementations produced identical outputs, and the self-check confirmed correctness:

Element-wise self-check: PASS

Output of test on node09 using small input:

```
build/part_baseline --N 16 --P 8 --seed 42 --small-check --print-sample
Element-wise self-check: PASS
Configuration:
  N      = 16
  P      = 8
  seed   = 42
  repeats = 5
Results:
  checksum      = 3372711514673124312
  median time (ms) = 0.000
  mean time (ms)  = 0.000
  stddev time (ms) = 0.000
  throughput (elem/s) = 326530612.24
Sample output:
i=0 key=13930160852258120406 part=3
i=1 key=11788048577503494824 part=5
i=2 key=13874630024467741450 part=7
i=3 key=2513787319205155662 part=6
i=4 key=16662371453428439381 part=1
i=5 key=1735254072534978428 part=4
i=6 key=10598951352238613536 part=7
i=7 key=6878563960102566144 part=2
i=8 key=5052085463162682550 part=5
i=9 key=7199227068870524257 part=6
build/part_autovec --N 16 --P 8 --seed 42 --small-check --print-sample
Element-wise self-check: PASS
```

```
build/part_autovec --N 16 --P 8 --seed 42 --small-check --print-sample
Element-wise self-check: PASS
Configuration:
  N      = 16
  P      = 8
  seed   = 42
  repeats = 5
Results:
  checksum      = 3372711514673124312
  median time (ms) = 0.000
  mean time (ms)  = 0.000
  stddev time (ms) = 0.000
  throughput (elem/s) = 410256410.26
Sample output:
i=0 key=13930160852258120406 part=3
i=1 key=11788048577503494824 part=5
i=2 key=13874630024467741450 part=7
i=3 key=2513787319205155662 part=6
i=4 key=16662371453428439381 part=1
i=5 key=1735254072534978428 part=4
i=6 key=10598951352238613536 part=7
i=7 key=6878563960102566144 part=2
i=8 key=5052085463162682550 part=5
i=9 key=7199227068870524257 part=6
```

4.2 Checksum Validation (Large Input)

For large datasets, where element-wise comparison is impractical, correctness is verified using a checksum computed over the output array:

checksum = hash(part_ids)

For the large benchmark configuration ($N = 50,000,000$, $P = 256$), all implementations produced the same checksum: **18151475427795858923**.

```
h.ali76@node09:~/Partition_hash_joined_Project/module_1$ make run-large
mkdir -p build reports
g++ -O3 -std=c++17 -Wall -Wextra -Wpedantic -mavx2 -fno-tree-vectorize src/main.cpp -o build/part_autovec
g++ -O3 -std=c++17 -Wall -Wextra -Wpedantic -mavx2 -fopt-info-vec-all=reports/vec-info.txt -o build/part_autovec
build/part_baseline --N 50000000 --P 256 --seed 42 --repeats 7
Configuration:
  N      = 50000000
  P      = 256
  seed   = 42
  repeats = 7
Results:
  checksum      = 18151475427795858923
  median time (ms) = 62.519
  mean time (ms)  = 62.521
  stddev time (ms) = 0.045
  throughput (elem/s) = 799756093.59
```

```
build/part_autovec --N 50000000 --P 256 --seed 42 --repeats 7
Configuration:
  N      = 50000000
  P      = 256
  seed   = 42
  repeats = 7
Results:
  checksum      = 18151475427795858923
  median time (ms) = 45.735
  mean time (ms)  = 45.645
  stddev time (ms) = 0.242
  throughput (elem/s) = 1093251822.23
h.ali76@node09:~/Partition_hash_joined_Project/module_1$
```

5. Performance Evaluation

5.1 Experimental Setup

- Platform: **node09 (SPM cluster)**
- Input size: $N = 50,000,000$
- Partitions: $P = 256$, Seed: 42
- Repetitions: 7

Metrics: median time, std dev, throughput (elem/s), speedup vs baseline vs AVX2 direct implementation

Version	Median Time (ms)	Std Dev (ms)	Throughput (elem/s)	Speedup
Baseline	61.959	0.099	8.07e8	1.00×
Auto-vectorized	47.180	0.414	1.06e9	1.31×
AVX2	46.508	0.866	1.08e9	1.33×

6. Performance Discussion

The baseline processes one element at a time in scalar form, achieving a throughput of **807 M elem/s** (median time: 61.959 ms). Since vectorization is explicitly disabled, the CPU executes one multiply, shift, and mask per clock cycle iteration, leaving the wide AVX2 execution units completely idle.

Enabling GCC auto-vectorization improves throughput to **1.06 B elem/s**, a **1.31× speedup**. This confirms that the loop in `map_keys_to_partitions` is well-suited for automatic vectorization: it has no loop-carried dependencies, uniform stride-1 memory access, and only data-parallel arithmetic. The compiler successfully emits 256-bit vector instructions, processing multiple elements per cycle.

The manual AVX2 version reaches **1.08 B elem/s**, a **1.33× speedup** over baseline, but only marginally better than auto-vectorization (+0.67 ms). This small gap shows that the compiler already generates near-optimal vector code for this kernel, and manual intrinsics offer little additional benefit.

The modest overall speedup (1.33× despite using 256-bit registers) is explained by two factors. First, the 64→32-bit XOR fold is performed in scalar form before loading values into vector registers, meaning this step does not benefit from SIMD and limits the effective parallelism. Second, the kernel has very low arithmetic intensity, approximately 4 operations per key (XOR, multiply, shift, mask) . so it reads and writes memory faster than it computes. The total data movement is ~600 MB for N = 50M (8 bytes read + 4 bytes written per key), and the AVX2 version achieves ~13 GB/s effective bandwidth. This is well below node09's peak memory bandwidth, indicating the bottleneck is the scalar folding overhead and instruction latency rather than memory saturation.

In summary, the kernel benefits significantly from vectorization over the scalar baseline, but further gains are constrained by the scalar folding step and low arithmetic intensity. Fully vectorizing the fold operating on 64-bit keys directly inside SIMD registers, would be the natural next optimization.